

AI FAQ ASSISTANT API

A RESTful AI-Powered Frequently Asked Questions Management System

Team ID: SWTID-2026-6721

Project Name: AI FAQ Assistant API

Technology: Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, Google Gemini AI

1. Brainstorming & Idea Prioritization

Step 1: Brainstorm and Idea Listing

S . N o	Team Member	Idea / Suggestion	Category
1	Team Member 1	AI-powered FAQ system	Artificial Intelligence
2	Team Member 2	Automated question-answer generation	AI / Automation
3	Team Member 3	Secure user authentication	Security
4	Team Member 4	FAQ management and search API	Web Development

Step 2: Final Idea

Final Project Idea:

Develop a RESTful API that allows users to register and authenticate securely, create and manage FAQs, search FAQ content, and use Google Gemini AI to generate answers and FAQ question-answer pairs.

2. Problem Statement

Users and support teams often spend time manually creating answers for frequently asked questions. Managing these questions, organizing them into categories, searching existing answers, and maintaining secure access can become difficult.

The proposed **AI FAQ Assistant API** provides a centralized backend solution for authentication, FAQ management, search, and AI-powered answer generation.

3. Empathy Map

Persona

Name: FAQ Content Creator

User Type: Authenticated User / Content Creator

SAYS

- "I need quick answers to common questions."
- "I want to create and manage FAQs easily."
- "I need secure access to my content."

THINKS

- "Can AI generate a useful answer?"
- "Can I find existing FAQs quickly?"
- "Is my account and data secure?"

DOES

- Registers and logs into the system.
- Creates and searches FAQs.
- Uses AI to generate answers.

FEELS

- Wants a simple and efficient system.
 - Feels confident when content is securely managed.
 - Saves time through AI assistance.
-

4. Customer Journey Map

Stage	Stage 1: Registration	Stage 2: FAQ Management	Stage 3: AI Assistance
Actions	Creates account and logs in	Creates, views, updates or deletes FAQs	Sends questions/topi cs to AI
Touchpoint	Authentication API	FAQ API	Gemini AI API
Thought	"My account should be secure."	"I need to manage my FAQs."	"I need a quick answer."
Feeling	Secure	Organized	Assisted
Process Ownership	Authentication module	FAQ controller/database	AI service
Opportunity	Improve authentication	Improve search and organization	Improve AI responses

5. Solution Requirements

5.1 Functional Requirements

S - N o	Requirement Category	Description	P r i o r i t y
1	Authentication	Users can register and log in using email and password.	H i g h

2	Authorization	JWT tokens protect private API endpoints.	H ig h
3	FAQ Management	Users can create, read, update and delete FAQs.	H ig h
4	Search	Users can search FAQs using keywords.	H ig h
5	AI Answer	Gemini AI generates answers to user questions.	H ig h
6	AI FAQ Generation	Gemini generates structured FAQ question-answer content.	H ig h
7	Category Management	FAQs are organized using predefined categories.	M e d i u m
8	Error Handling	API provides standardized error responses.	H ig h

5.2 Non-Functional Requirements

Category	Requirement	Target
Performance	API should respond efficiently to normal requests.	Fast response
Security	Passwords must not be stored as plain text.	bcrypt hashing
Authentication	Private routes require valid JWT tokens.	Token verification

Reliability	Server should handle invalid requests without crashing.	Centralized error handling
Maintainability	Code should be organized into modules.	MVC structure
Scalability	System should support increasing FAQ records and users.	Modular architecture

6. Technology Stack

Component	Technology	Purpose
Runtime	Node.js	Executes backend JavaScript
Backend	Express.js	Creates REST API endpoints
Database	MongoDB	Stores users and FAQs
ODM	Mongoose	Database schema and validation
Authentication	JWT	Secure token-based authentication
Password Security	bcrypt	Password hashing
AI	Google Gemini	AI-generated answers and FAQs
API Testing	Thunder Client / Postman	Testing API endpoints
Code Editor	Visual Studio Code	Development environment
Version	Git & GitHub	Source-code management

Control		
---------	--	--

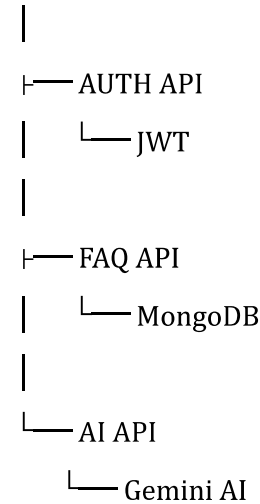
7. Project Architecture

The system follows a modular backend architecture.

CLIENT



EXPRESS.JS API



The Express.js server receives client requests and routes them to authentication, FAQ, or AI services. MongoDB stores application data, while Gemini AI handles AI-powered requests.

8. Data Flow Diagram

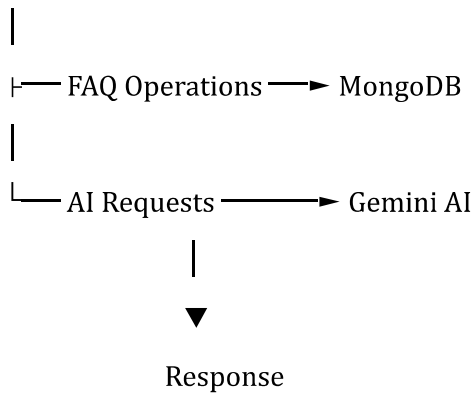
The main data flow is:

User



Express.js API





The API processes incoming requests, validates them, communicates with the required service, and returns a JSON response to the client.

9. Entity-Relationship Description

User Entity

- `_id`
- `name`
- `email`
- `password`
- `createdAt`
- `updatedAt`

FAQ Entity

- `_id`
- `question`
- `answer`
- `category`
- `createdBy`
- `createdAt`
- `updatedAt`

Relationship

User → FAQ

One registered user can create multiple FAQ records.

10. Key Features

- User registration
 - User login
 - JWT authentication
 - Secure password hashing using bcrypt
 - User profile access
 - FAQ creation
 - FAQ retrieval
 - FAQ updating
 - FAQ deletion
 - FAQ search
 - Category validation
 - AI-generated answers
 - AI-generated FAQ question-answer pairs
 - MongoDB database integration
 - Centralized error handling
-

11. User Flow

User



Register



Password Hashed



MongoDB



Login



Credentials Verified

|



JWT Token Generated

|



Client Uses Token

|



Protected API Request

|



JWT Verification

|



Access Granted

12. MVC Architecture

Model

Defines MongoDB data structures using Mongoose.

Examples:

- `User.js`
- `FAQ.js`

Controller

Handles application logic and processes API requests.

Examples:

- `authController.js`
- `faqController.js`
- `aiController.js`

Routes

Defines API endpoints and connects them to controllers.

Examples:

- `authRoutes.js`
- `faqRoutes.js`
- `aiRoutes.js`

Services

Handles external AI integration.

`geminiService.js` communicates with Google Gemini AI.

13. Project Setup

Create Project

```
mkdir AI-FAQ-Assistant
```

```
cd AI-FAQ-Assistant
```

```
npm init -y
```

Install Dependencies

```
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

Run Project

```
npm start
```

The API runs locally on:

```
http://localhost:5000
```

 **Attach Screenshot Here:** VS Code terminal showing the server running successfully.

14. Environment Configuration

Create a `.env` file:

```
PORT=5000
```

MONGO_URI=mongodb://127.0.0.1:27017/ai_faq_assistant

JWT_SECRET=your_secret_key

GEMINI_API_KEY=your_gemini_api_key

Important: Do not include the real Gemini API key or JWT secret in the documentation or GitHub repository. Use `.env.example` for public projects.

 **Attach Screenshot Here:** `.env` file with the secret values hidden/blurred.

15. API Testing

15.1 Register

Endpoint

POST /api/auth/register

Purpose: Creates a new user account.

 **Attach Screenshot:** Register request + successful response.

15.2 Login

Endpoint

POST /api/auth/login

Purpose: Verifies user credentials and generates a JWT token.

 **Attach Screenshot:** Login request + JWT response.

15.3 Profile

Endpoint

GET /api/auth/profile

Purpose: Retrieves information about the authenticated user.

 **Attach Screenshot:** Profile request with `Authorization: Bearer <token>` + response.

15.4 Create FAQ

Endpoint

POST /api/faqs

Purpose: Creates a new FAQ.

 **Attach Screenshot:** POST request + successful FAQ response.

15.5 Get FAQs

Endpoint

GET /api/faqs

Purpose: Retrieves available FAQs.

 **Attach Screenshot:** GET request + FAQ list response.

15.6 Update FAQ

Endpoint

PUT /api/faqs/:id

Purpose: Updates an existing FAQ created by the authenticated user.

 **Attach Screenshot:** PUT request + updated response.

15.7 Delete FAQ

Endpoint

DELETE /api/faqs/:id

Purpose: Deletes an FAQ owned by the authenticated user.

 **Attach Screenshot:** DELETE request + successful response.

15.8 Search FAQ

Endpoint

GET /api/faqs/search?q=keyword

Purpose: Searches FAQ questions, answers, and categories.

 **Attach Screenshot:** Search request + matching results.

15.9 AI Answer

Endpoint

POST /api/ai/answer

Purpose: Uses Google Gemini AI to generate an answer to a user question.

 **Attach Screenshot:** Question request + generated AI answer.

15.10 AI FAQ Generation

Endpoint

POST /api/ai/generate-faq

Purpose: Generates a structured FAQ question and answer based on a given topic.

 **Attach Screenshot:** Topic request + generated question and answer.

16. Communication Plan

Communication Type	Frequency	Channel	Participants	Purpose
Team Discussion	Daily	WhatsApp	All members	Discuss progress

Progress Update	Weekly	WhatsApp / Meet	All members	Track development
Issue Discussion	As Needed	WhatsApp	Relevant members	Solve technical issues
Code Collaboration	As Needed	GitHub	Development team	Manage source code
Demo Rehearsal	Once	College / Online	All members	Prepare final demonstration

17. Demonstration of Proposed Features

Feature	Description	Status	Demonstrated
User Registration	Creates a new account	Implemented	Yes
User Login	Authenticates users and generates JWT	Implemented	Yes
User Profile	Displays authenticated user information	Implemented	Yes
FAQ CRUD	Create, read, update and delete FAQs	Implemented	Yes
FAQ Search	Searches FAQs using keywords	Implemented	Yes
AI Answer	Generates answers using Gemini	Imple	Yes

	AI	mente d	
AI FAQ Generation	Generates FAQ question-answer pairs	Imple mente d	Yes

18. Project Demo Planning

Demo Section	Description	Responsible Member
Introductio n	Introduce project and problem	Team Leader
Architectur e	Explain system architecture	Member 2
Authenticat ion	Demonstrate Register, Login and Profile	Member 3
FAQ Manageme nt	Demonstrate CRUD and Search	Member 4
AI Features	Demonstrate Gemini AI features	Member 1
Conclusion	Explain outcome and future scope	Team Leader

19. Scalability & Future Plan

Current Limitations

Limitation	Impact
Local MongoDB setup	Requires local database configuration

AI depends on external Gemini service	AI features require API access
Backend-only system	Requires a separate frontend for a complete user interface

Future Enhancements

Area	Proposed Enhancement
Front end	Build a React-based user interface
Deployment	Deploy API using a cloud platform
Database	Use MongoDB Atlas
Search	Improve search using semantic/vector search
Security	Add rate limiting and additional security controls
AI	Improve prompts and AI response handling

20. Team Involvement

Team Member	Role	Contribution
Member 1	Team Leader / Backend	Project coordination and backend development
Member 2	Developer	Authentication and security
Member	Developer	FAQ and database functionality

3		
Member 4	Developer	AI integration and testing

Replace **Member 1–4** with your actual four team members' names.

Project Structure

AI-FAQ-Assistant/

|

├── src/

|

├── config/

|

├── controllers/

|

├── middleware/

|

├── models/

|

├── routes/

|

├── services/

|

├── utils/

|

├── app.js

|

└── server.js

|

├── .env.example

├── .gitignore

├── package.json

├── package-lock.json

|

└── README.md

22. Run Instructions

Step 1

Open the project folder in VS Code.

Step 2

npm install

Step 3

Create the `.env` file and configure:

PORT

MONGO_URI

JWT_SECRET

GEMINI_API_KEY

Step 4

Start MongoDB.

Step 5

Run:

npm start

Step 6

Open Thunder Client or Postman.

Step 7

Test the authentication, FAQ, search and AI endpoints.

 **Attach Screenshot:** Terminal showing:

MongoDB Connected

Server running on port 5000

23. Known Issues / Limitations

Issue

Workaround / Status

Gemini API requires a valid API key

Configure `GEMINI_API_KEY` in `.env`

MongoDB must be running

Start MongoDB before running the API

Protected APIs require JWT

Login first and use the generated token

No frontend interface

API can be tested through Postman/Thunder Client

24. Conclusion

The **AI FAQ Assistant API** provides a secure and modular backend for managing frequently asked questions. It combines RESTful API development, MongoDB data management, JWT authentication, bcrypt password protection, and Google Gemini AI to provide automated question-answer functionality.

The project demonstrates how backend technologies and AI services can be integrated to create an efficient FAQ management system with scope for future frontend development, cloud deployment, and advanced AI-powered search.